certora

Security Assessment & Formal Verification Report

Reserve

January 2026

Prepared for Reserve

certora

Table of Contents

Project Summary

Project Scope

Project Name	Repository (link)	First Commit Hash	Latest Commit Hash	Platform
Reserve Protocol	Repo	7336225		EVM

Project Overview

This document describes the specification and verification of **Reserve Protocol** using the Certora Prover. The work was undertaken from **December 22nd, 2025** to **January 12th, 2026**

The following library is included in our scope:

- contracts/library/Fixed.sol

The Certora Prover demonstrated that the implementation of the **Solidity** library above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solidity library code, as listed on the following pages.

Protocol Overview

The Fixed library is a fixed-point arithmetic library for Solidity that represents decimal values using a custom 192-bit unsigned type (uint192) with 18 decimal places (i.e., scaled to 1e18). It provides safe conversions and common operations such as addition/subtraction, multiplication/division with configurable rounding, exponentiation and square root, as well as "safe" variants that saturate instead of reverting - while reverting when results exceed the representable uint192 range.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	2	2	2
Informational	3	3	0
Total	6	6	3

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	safeMulDiv() may return 0 instead of FIX_MAX.	Medium	Fixed
L-01	safeDiv() doesn't correctly propagate the FIX_MAX value.	Low	Fixed
L-02	mul() round differently from other functions by default.	Low	Fixed

Medium Severity Issues

M-01 safeMulDiv() may return 0 instead of FIX_MAX.

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: Fixed.sol	Status: Fixed	

Description:

In line 592, result_256 holds the rounded down result of `safeMulDiv()`, which will not overflow the uint256 from the check in line 573, "if (hi >= c) return FIX_MAX;". When rounding ROUND or CEIL, if result_256 is $2^{256} - 1$, we may round up to 2^{256}. As this is unchecked, it will be rounded down to 0, and returned.

Recommendations: Add the check "if (result_256 >= FIX_MAX) return FIX_MAX;" before applying rounding, or add a check for the case result_256 is $2^{256} - 1$.

Customer's response: Acknowledged, fixed in 41136fd.

Fix Review: Fixed confirmed.

Low Severity Issues

L-01 safeDiv() doesn't correctly propagate the FIX_MAX value.

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: Fixed.sol	Status: Fixed	Violated Property: safeDiv correctness

Description: The function `safeDiv()` doesn't propagate the value FIX_MAX. So a call to safeDiv(FIX_MAX, 2), will return the value FIX_MAX/2 and not FIX_MAX as it should. We note that the function `safeMul()` does propagate FIX_MAX, so for example `safeMax(FIX_MAX,½)` returns FIX_MAX.

Recommendations: Align the behaviour of `safeDiv()` to the one of `safeMul()`.

Customer's response: Acknowledged, fixed in 7a2ca79.

Fix Review: Fixed confirmed.

L-02 mul() round differently from other functions by default.

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: Fixed.sol	Status: Fixed	

Description:

The non-rounding version of the `mul()` function uses ROUND (natural rounding) as the default rounding behavior (line 253), as opposed to FLOOR used by the other non-rounding function variants.

Recommendations: Align the rounding behaviour of the `mul()` function to use FLOOR.

Customer's response: Acknowledged, fixed in 7a2ca79.

Fix Review: Fixed confirmed.

Informational Issues

I-01. Incorrect comments.

Description:
In the code there are a number of incorrect and unclear comments:

a) Line 102 (`shiftl_toFix()`) - The comment says "// 0 < uint.max / 10**77 < 0.5", which is incorrect, as uint.max / 10^{77} is 1.11579... .
b) Line 114 (`divFix()`) - The comment says "`This may also fail if the result is MIN_uint192! not fixing this for optimization's sake.`", but the problem here is if we divide by 0 (y = 0).
c) Line 213 (`shiftl()`) - The comment says "`// 59, because 1e58 > 2**192`", which explains checking the decimals against -58 and not -59.
d) Lines 498 to 539 - `safeMul()` - There are lots of comments referencing code outside the library, and explaining operations not happening in this function.
e) Line 661 (`mulDiv256()`) - The comment says "`// z should be z-1`", but z-1 is already used.
f) Line 714 (`sqrt256()`) - The comments says "`// 2^{log_2(x)} <= x <= 2*2^{log_2(x)+1} \\`", but the right hand size should be 2 times the left hand size. Only one of "2*" and "+1" are needed.

Recommendation:

a) The correct statement is "uint.max / 10^78 (< 0.12) < 0.5".
b) The comment should state the problem is with division by 0, and not a 0 result.
c) The check against -59 should be explained.
d) The comments in the `safeMul()` function should be cleaned up and rewritten.
e) The comment should be erased
f) Only one of "2*" and "+1" should remain, the other should be erased.

Customer's response: Acknowledged, will fix in future release.

Fix Review: Not fixed in current version.

I-02. Unused variable.

Description:
In line 311, the variable “FIX_HALF” is declared but never used.

Recommendation:
Erase this declaration.

Customer’s response: Acknowledged, will fix in future release.

Fix Review: {Comments about the fix}

I-03. The behavior of 0/0 in safe functions is undocumented.

Description:
`safeDiv()` and `safeMulDiv()` need to return a value for 0/0, and they return 0 in this case. While this choice is alright, it should be explicitly documented, and it is not the only possible choice for this edge case.

Recommendation:
Add comments explaining this case at the beginning of the function.

Customer’s response: Acknowledged, will fix in future release.

Fix Review: Not fixed in current version.

Formal Verification

Verification Methodology

We performed verification of the **Reserve** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and **Reserve's** implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

Formal Verification Properties Overview

ID	Title	Status
P-01	sqrt256() correctness	Verified
P-02	mulDiv256() correctness	Verified
P-03	powu() correctness	Verified
P-04	toFix() correctness	Verified
P-05	shiftl_toFix() correctness	Verified
P-06	divFix() correctness	Verified
P-07	divuu() correctness	Verified
P-08	toUint() correctness	Verified
P-09	shiftl() correctness	Verified
P-10	plus() correctness	Verified
P-11	plusu() correctness	Verified

P-12	minus() correctness	Verified
P-13	minusu() correctness	Verified
P-14	mul() correctness	Verified
P-15	mulu() correctness	Verified
P-16	div() correctness	Verified
P-17	divu() correctness	Verified
P-18	shiftl_toUint() correctness	Verified
P-19	mulu_toUint() correctness	Verified
P-20	mul_toUint() correctness	Verified
P-21	muluDivu() correctness	Verified
P-22	mulDiv() correctness	Verified
P-23	safeMul() correctness	Verified
P-24	safeDiv() correctness	Violated. Issue L-01

P-25	fullMul() correctness	Verified

Detailed Properties

P-01. sqrt256() correctness

Status: Verified

High-level description:
For any input x, we prove that sqrt256(x) returns the integer square root of x, i.e. a value val such that: $val^2 \leq x < (val + 1)^2$.
The proof is structured as a chain of CVL rules that mirror the implementation. Each rule establishes that the intermediate variable result satisfies progressively tighter bounds around √x after each phase (initial guess, each Newton iteration, and the final rounding step), until the final rule yields the two defining inequalities above.

The implementation of sqrt256 uses the Babylonian (Heron's) method for computing square roots - equivalently, Newton's method applied to $r^2 - x$. The CVL proof is structured around per-iteration invariants: after the initial estimate, each Newton update refines result, and we carry forward progressively tighter bounds derived from the method's standard convergence analysis - specifically the relative-error recurrence - until the final rounding-down step yields $val^2 \leq x < (val + 1)^2$. For background on the method and its convergence/error behavior, see the "Heron's method" section on Wikipedia.

In the below description of the rules, we use x for the input, and r for the variable result.

Rule Name	Status	Description	Link to rule report
sqrt256_Part1 ValidOutput	Verified	*Validate that after the first part of the function, i.e. after the "if (xAux >= 2**2) {result <<= 1;}", it holds that:* $r^2 \leq x < 4 \cdot r^2$	*report*
sqrt256_Part2 _Iter_1	Verified	*Validates that after the first execution of "result = (result + x / result) >> 1;" it holds that:* $r^2 \leq \frac{25 \cdot x}{16}$ *and* $x < (r + 1)^2$	
sqrt256_Part2 _Iter_2	Verified	*Validates that after the second execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r + 1)^2$ *and* $(r - 1) \cdot r \leq x$ *or* $r^2 \leq x + \frac{x}{81}$	

sqrt256_Part2_Iter_3	Verified	*Validates that after the third execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r+1)^2$ *and* $(r-1)\cdot r \le x$ *or* $r^2 \le x + \frac{x}{1639}$
sqrt256_Part2_Iter_4	Verified	*Validates that after the forth execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r+1)^2$ *and* $(r-1)\cdot r \le x$ *or* $r^2 \le x + \frac{x}{10751840}$
sqrt256_Part2_Iter_5	Verified	*Validates that after the fifth execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r+1)^2$ *and* $(r-1)\cdot r \le x$ *or* $r^2 \le x + \frac{x}{462408296549760}$
sqrt256_Part2_Iter_6	Verified	*Validates that after the second execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r+1)^2$ *and* $(r-1)\cdot r \le x$ *or* $r^2 \le x + \frac{x}{855285730872204993313810429440}$
sqrt256_Part2_Iter_7	Verified	*Validates that after the second execution of "result = (result + x / result) >> 1;" it holds that:* $x < (r+1)^2$ *and* $(r-1)\cdot r \le x$ *or* $r^2 \le x + \frac{x}{2926054725734407478371345979251228656221734313834524116572160}$
sqrt256_Part3	Verified	*Validate that at the end of the function, the returned value r satisfy:* $r^2 \le x$ *and* $x < (r+1)^2$

P-02. mulDiv256() correctness

Status: Verified

High-level description:
For any inputs x,y,z, we prove that mulDiv256(x,y,z) returns $\frac{x \cdot y}{z}$ (assuming it fits into a uint256).
For that we break the function mulDiv256() into the following 4 phases, and prove several properties about the variables after each phase:
phase 1: substractReminer. Up to the line "lo -= mm;".
phase 2: factorPowersOfTwo. From the end of phase-1 up to the line "lo += hi * ((0 - pow2) / pow2 + 1);"
phase 3: multiplicativeInverse. From the end of phase-2 up to the last line "r *= 2 - z * r;".
phase 4: finalPhase. The line "result = lo * r;"

Rule Name	Status	Description	Link to rule report
mulDivBoundsCheck	Verified	*Validate that z > 0 and (x*y)/z < 2^256, during the entire computation.*	*report*
subtractRemainderCheck	Verified	*Validates that by the end of phase-1 the following holds:* - *(x * y) / z == (lo + hi * 2^256) / z* - *(lo + hi * 2^256) % z == 0* - *(lo + hi * 2^256) / z < 2^256*	
factorPowersOfTwo_result	Verified	*Validates that by the end of phase-2 the following hold:* - *z % 2 == 1* *and if we denote by old_<var> the values of <var> at the end of phase-1:* - *(old_lo + old_hi * 2^256) / old_z == (lo + hi * 2^256) / z* - *(lo + hi * 2^256) % z == 0*	
multiplicativeInverseStepCheck	Verified	*We want to prove that after phase-3 the following holds:* - *(r * z) % 2^256 == 1* *To do that we prove by induction on i (for i = 0 to 8) that the following holds:* *(r*z) % 2^(2^i)==1* *The base is trivial, because for i=0 we have (1*z) % 2 == 1 because z is odd. The induction step is proved in this rule.*	
multiplicativeInverseApplication	Verified	*Validates that at the end of the function:* *- (lo * r) % 2^256 == (lo + hi * 2^256) / z* *Since the return value is (lo * r) % 2^256, and since (lo + hi * 2^256) / z equals to the (x*y)/z (where x,y,z are the inputs), this last rule finishes the correctness proof of the function.*	

P-03. powu() correctness

Status: Verified	**High-level description:** Assuming $x \leq 10^{18}$ (namely, it represent a number less or equal to 1) it holds that: $$\left\|powu(x,y) - \frac{x^y}{10^{18(y-1)}}\right\| \leq 1$$ We provide a standard mathematical proof for the above. See the Appendix.

P-04. toFix() correctness

Status: Verified	**High-level description:** Prove that toFix(x)==x*1e18, and reverts only if it can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
toFix_correctness	Verified	*See the above high-level description*	*report*

P-05. shiftl_toFix() correctness

Status: Verified	**High-level description:** Prove that shiftl_toFix(x,shiftLeft,rounding)==x * 10**(shiftLeft + 18) (taking into account rounding issues), and reverts only if it can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
shiftl_toFix_correctness	Verified	*See the above high-level description*	*report*

P-06. divFix() correctness

Status: Verified	**High-level description:** Prove that divFix(x,y)==x * 1e36 / y, and reverts only if y==0 or result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
divFix_correctness	Verified	*See the above High-level description*	*report*

P-07. divuu() correctness

Status: Verified	**High-level description:** Prove that divuu(x,y)==x * 1e18 / y, and reverts only if y==0 or result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
divuu_correctness	Verified	*See the above High-level description*	*report*

P-08. toUint() correctness

Status: Verified	**High-level description:** Prove that toUint(x,rounding)==x / 1e18 (with rounding), and never reverts.

Rule Name	Status	Description	Link to rule report

toUint_correctness	Verified	*See the above High-level description*	*report*

P-09. shiftl() correctness

Status: Verified	**High-level description:** Prove that shiftl(x,decimals,rounding)==x * 10**decimals (with rounding), and reverts only if the result can't fit into uint192 type.

Rule Name	Status	Description	Link to rule report
shiftl_correctness	Verified	*See the above High-level description*	*report*

P-10. plus() correctness

Status: Verified	**High-level description:** Prove that plus(x,y)==x + y, and revert only if result can't fit into uint192 type.

Rule Name	Status	Description	Link to rule report
plus_correctness	Verified	*See the above High-level description*	*report*

P-11. plusu() correctness

Status: Verified	**High-level description:** Prove that plusu(x,y)==x + y*1e18, and reverts only if the result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
plusu_correctness	Verified	*See the above High-level description*	*report*

P-12. minus() correctness

Status: Verified	**High-level description:** Prove that minus(x,y)==x - y, and reverts only if x < y.

Rule Name	Status	Description	Link to rule report
minus_correctness	Verified	*See the above High-level description*	*report*

P-13. minusu() correctness

Status: Verified	**High-level description:** Prove that minusu(x,y)==x - y*1e18, and reverts only if x < y*1e18.

Rule Name	Status	Description	Link to rule report
minusu_correctness	Verified	*See the above High-level description*	*report*

P-14. mul() correctness

Status: Verified	**High-level description:** Prove that mul(x,y,rounding)==x * y / 1e18 (with rounding), and reverts only if the result can't fit into uint192 type.

Rule Name	Status	Description	Link to rule report
mul_correctness	Verified	*See the above High-level description*	*report*

P-15. mulu() correctness

Status: Verified	**High-level description:** Prove that mulu(x,y)==x * y, and reverts only if the result can't fit into uint192 type.

Rule Name	Status	Description	Link to rule report
mulu_correctness	Verified	*See the above High-level description*	*report*

P-16. div() correctness

Status: Verified	**High-level description:** Prove that div(x,y,rounding)==x * 1e18 / y (with rounding), and reverts only if y==0 or result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
div_correctness	Verified	*See the above High-level description*	*report*

P-17. divu() correctness

Status: Verified	**High-level description:** Prove that divu(x,y,rounding)==x / y (with rounding), and reverts only if y==0.

Rule Name	Status	Description	Link to rule report
divu_correctness	Verified	*See the above High-level description*	*report*

P-18. shiftl_toUint() correctness

Status: Verified	**High-level description:** Prove that shiftl_toUint(x,decimals,rounding)==x * 10**(decimals - 18) (with rounding), and reverts only if the result can't fit into uint256 type.

Rule Name	Status	Description	Link to rule report
shiftl_toUint_correctness	Verified	*See the above High-level description*	*report*

P-19. mulu_toUint() correctness

Status: Verified	**High-level description:**

	Prove that mulu_toUint(x,y,rounding)==x * y / 1e18 (with rounding), and reverts only if the result can't fit into uint256 type.

Rule Name	Status	Description	Link to rule report
mulu_toUint_correctness	Verified	*See the above High-level description*	*report*

P-20. mul_toUint() correctness

Status: Verified	**High-level description:** Prove that mul_toUint(x,y,rounding)==x * y / 1e36 (with rounding), and reverts only if the result can't fit into the uint256 type.

Rule Name	Status	Description	Link to rule report
mul_toUint_correctness	Verified	*See the above High-level description*	*report*

P-21. muluDivu() correctness

Status: Verified	**High-level description:** Prove that muluDivu(x,y,z,rounding)==x * y / z (with rounding), and reverts only if z==0 or result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
muluDivu_correctness	Verified	*See the above High-level description*	*report*

P-22. mulDiv() correctness

Status: Verified	**High-level description:** Prove that mulDiv(x,y,z,rounding)==x * y / z (with rounding), and reverts only if z==0 or result can't fit into the uint192 type.

Rule Name	Status	Description	Link to rule report
mulDiv_correctness	Verified	*See the above High-level description*	*report*

P-23. safeMul() correctness

Status: Verified	**High-level description:** Prove that safeMul(a,b,rounding)==a * b / 1e18 (with rounding), saturating to FIX_MAX instead of reverting on overflow, and never reverts.

Rule Name	Status	Description	Link to rule report
safeMul_correctness	Verified	*See the above High-level description*	*report*

P-24. safeDiv() correctness

Status: Verified	**High-level description:** Prove that safeDiv(a,b,rounding)==a * 1e18 / b (with rounding), saturating to FIX_MAX instead of reverting on overflow or division by zero, and never reverts.

Rule Name	Status	Description	Link to rule report

safeDiv_correctness	Violated	*See L-01*	*report*

P-25. fullMul() correctness

Status: Verified	**High-level description:** Prove that fullMul(x,y) returns (hi, lo) such that hi*(2**256) + lo == x * y, and never reverts.

Rule Name	Status	Description	Link to rule report
fullMul_correctness	Verified	*See the above High-level description*	*report*

Test Suite Coverage

The following functions are not directly covered by tests:

None

```
function _safeWrap(uint256 x)
function divuu(uint256 x, uint256 y)
function sqrt256(uint256 x)
```

The following functions are not directly covered by fuzz tests:

None

```
function _safeWrap(uint256 x)
function fixMin(uint192 x, uint192 y)
function fixMax(uint192 x, uint192 y)
function sqrt(uint192 x)
function lt(uint192 x, uint192 y)
function lte(uint192 x, uint192 y)
function gt(uint192 x, uint192 y)
function gte(uint192 x, uint192 y)
function eq(uint192 x, uint192 y)
function neq(uint192 x, uint192 y)
function near(uint192 x, uint192 y, uint192 epsilon)
function safeMul(uint192 a, uint192 b, RoundingMode rounding)
function safeDiv(uint192 a, uint192 b, RoundingMode rounding)
function safeMulDiv(uint192 a,uint192 b, uint192 c, RoundingMode rounding)
function mulDiv256(uint256 x, uint256 y, uint256 z, RoundingMode rounding)
function sqrt256(uint256 x)
```

General test suggestions

Note: problems mentioned here are not rementioned in the specific suggestions part, but all locations where they are relevant are listed here.

Unverified rounding

Some tests calculate the result depending on the rounding used. This usually does not verify the result against an expected value. Adding this verification can give an additional sanity check on the tests.
Lines: 243-245, 273-275, 543-545, 640-642, 721-723, 950-952, 974-976.

Tests including non-trivial operations on MIN_UINT192

MIN_UINT192 is 0, and some tests check, for example, that MIN_UINT192/2 + MIN_UINT192/2 = MIN_UINT192. Consider instead checking similar checks with MAX_UINT192, and changing these tests to explicitly test 0.
Lines: 362, 438-440, 527-528, 580, 582, 634.

Adding commutations of test cases before adding edge cases

In some places, there is a commutation of the test cases. In most of these, after adding the commutated versions, edge cases are added. These edge cases can usually also be commuted.
Lines: 349 (`plus()`), 423 (`minus()`), 520-522 (`mul()`, `safeMul()`), 612 (`div()`, `safeDiv()`).

Some rounding tests use a smaller table

Many rounding checks are done using a small table. Consider using the original tables. Similarly, some rounding checks are not checked for general edge cases.
Lines: 539 (`mul()`, `safeMul()`), 632 (`div()`, `safeDiv()`), 713 (`divu()`).

Consider adding tests for rounding on the edge of the UINT192 range

Many functions with rounding return a fixed point number in uint192. Consider adding a case where the result rounds down within range and up outside the range.
Lines: 539 (`mul()`, `safeMul()`), 632 (`div()`, `safeDiv()`), 713 (`divu()`).

Tests in unexpected places

The fuzzing test for the `powu()` function is at the end of the fuzzing file, instead of in function order like the other fuzzing tests.
At the end of the fuzzing file, there are tests for `shiftl_toFix()` and `shiftl()` which seem to fit regular, and not fuzzing, tests.
In line 1160 there are tests for a "_safeDiv" or "safeDiv_" function - but no such function exists.

Test suggestions for specific functions

_safeWrap()

Consider checking the cases: 0, FIX_MAX-1, FIX_MAX, FIX_MAX+1, UINT256_MAX, and some arbitrary values between 0, FIX_MAX-1 and between FIX_MAX+1, UINT256_MAX.

divuu()

Consider adding tests similarly to that of other division functions (with an adapted table).

_divrnd()

Consider adding tests for edge cases.

shiftl()

Relevant to both rounding and non-rounding variants.
The current tests check that `toFix()` then `shiftl()` acts the same as `toFix_shiftl()`. While this is great, consider also testing the function directly.

minusu()

The current 'correctly subtracts at the extremes of its range' tests only check subtracting 0 from MAX_UINT192. Consider also adding additional subtractions from MAX_UINT192 (such as by 1, MAX_UINT192-1, MAX_UINT192).
In the 'fails outside its range' part, consider subtracting MAX_UINT192/SCALE + 1 from MAX_UINT192.

powu()

The fuzzing test has some problems - there is no filtering of base>fp(1) cases, which the function doesn't handle. It is also unclear why an error of 1 is expected in any direction - if this is required, the reasoning should be documented.
Consider planning a test that will validate the accuracy of the function (i.e. will round extremely in intermediate results in the loop).

sqrt()

Consider adding test cases which get a <fp(1) input in the "main" test table.
Consider adding roots of non-whole-squares, and checking that the rounding is correct.

near()

Consider adding to the table the expected truth value as an additional sanity check.

shiftl_toUint()

Relevant to both rounding and non-rounding variants.
Currently, only 0 and negative decimals are being tested. Consider adding test cases for positive decimals, and including checks for overflow when receiving positive decimals.
Consider checking the operation is equivalent to composing the underlying operations `shiftl()`+`toUint()`.

mulu_toUint() and mul_toUint()

Relevant to both rounding and non-rounding variants.
These functions are all not checked on edge cases - consider adding these.
Consider checking the operation is equivalent to composing the underlying operations `mul()`/`mulu()`+`toUint()`.

muluDivu(), mulDiv() and mulDiv256()

Relevant to both rounding and non-rounding variants.
Currently only (0,0,1) cases (`muluDivu()`, `mulDiv256()`), or (0,0,*) cases (`mulDiv()`), are being tested. Consider adding other cases, such as those covered by the `safeMulDiv()` tests.

safeMul() and safeDiv()

Consider documenting that the tests are split between two parts of the file, so that in future test reviews existing tests will not be missed.

safeMulDiv()

Some tests under 'resolves to FIX_MAX' do not resolve to FIX_MAX. Consider putting these tests under a different label.
Consider adding tests that divide by zero and do not independently multiply to FIX_MAX.
Consider adding some tests for “regular” cases.

fullMul()

Currently, only (0,0) is being tested.
Consider adding a variation of the “`mulu_table`”.
Consider adding additional “regular” cases, with some results being $<2^{256}$, and some being $>2^{256}$.
Consider adding checks for edge-cases: what happens when the multiplication results in lo=0, what happens when lo>mod(2^{256}-1), lo<mod(2^{256}-1) (maybe also hi>mod(2^{256}-1), hi<mod(2^{256}-1)).

sqrt256()

Consider adding tests similar to `sqrt()`.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response

certora

Appendix.